HTTP Client

Small HTTP requests, directly from DuckDB.

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL http_client FROM query_farm;  LOAD http_client FROM query_farm;
```

01 At a glance

Fetch HTTP resources or submit JSON and form bodies from a SQL expression, through DuckDB's own HTTP stack. Responses arrive as a JSON envelope containing status, reason, headers and body, ready to inspect or save in a local table.

HTTP request → JSON envelope → SQL projection

02 Requests & response fields

`http_get(url)`

Fetch a response; headers and params are optional, by name.

`http_head(url)`

Request metadata only; headers return as a JSON object.

`http_post(url, headers, body)`

Send a JSON body; headers are MAP(VARCHAR, VARCHAR).

`http_put(url, headers, body)`

Replace a resource; http_delete(url, headers) removes one.

`http_post_form(url, headers, params)`

Send form fields supplied as a VARCHAR-to-VARCHAR map.

`response->>'status'`

Read the status; ->>'body' extracts the body text.

`response->'headers'->>'content-type'`

Read one response header from the JSON object.

`SET http_client_max_response_size`

Cap bodies (100 MiB); http_client_max_redirects defaults to 10.

03 Keep in mind

DuckDB's HTTP stack

https:// and methods other than plain GET need httpfs (autoloaded). http_timeout, http_retries, http_proxy and HTTP secrets apply. Nothing is sent while enable_external_access is false; https:// never redirects to http://.

04 Fetch once, inspect locally

REQUIRES SETUP

```
CREATE TEMP TABLE response AS
SELECT http_get('https://example.com/') AS r;
SELECT (r->>'status')::INTEGER AS status,
       r->>'reason' AS reason,
       r->>'body' AS body
FROM response;
```

Requires network access. The example.com response is HTML, so keep body as text.

05 POST a JSON document

REQUIRES SETUP

```
SELECT http_post(
  'https://api.example.com/events',
  MAP {'content-type': 'application/json'},
  '{"event":"ready","rows":42}'::JSON
);
```

Replace the URL with your POST endpoint and add required authentication headers.

06 Decode a JSON response body

```
WITH r AS (
  SELECT '{"status":200,
    "body":"{\"count\":42}"}'::JSON AS v
)
SELECT ((v->>'body')::JSON)->>'count' AS n
FROM r;
```

Offline envelope fixture; returns the text value 42.

07 In practice

Inspect status before parsing

Requests are synchronous; save responses to a table before repeated analysis. A body can be HTML, text or JSON, and non-UTF-8 bodies arrive base64-encoded with body_encoding. Check status before casting; an HTTP error is not an API result.

Full docs & examples ↗ Source ↗